

TRIGGERS AND PROCEDURES

1.

```
CREATE TABLE Department_Stats (  
    dept VARCHAR(50) PRIMARY KEY,  
    count INT DEFAULT 0  
);  
  
CREATE TABLE Employees (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    dept VARCHAR(50),  
    hired DATE,  
    FOREIGN KEY (dept) REFERENCES Department_Stats(dept)  
);  
  
INSERT INTO Department_Stats (dept, count) VALUES  
('Engineering', 0),  
('Marketing', 0),  
('Human Resources', 0);  
  
DELIMITER //  
  
CREATE TRIGGER After_Employee_Insert  
AFTER INSERT ON Employees  
FOR EACH ROW  
BEGIN  
    UPDATE Department_Stats  
    SET count = count + 1  
    WHERE dept = NEW.dept;  
END //  
  
DELIMITER ;  
  
INSERT INTO Employees (name, dept, hired) VALUES  
('Pratham', 'Engineering', '2026-03-15'),  
('Shreya', 'Engineering', '2026-04-01'),  
('haha', 'Marketing', '2026-04-12');  
  
SELECT * FROM Department_Stats;
```

Output :

Output:

```
+-----+-----+
| dept          | count |
+-----+-----+
| Engineering   | 2     |
| Human Resources | 0     |
| Marketing     | 1     |
+-----+-----+
```

2.

```
CREATE TABLE Employees (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    salary INT NOT NULL  
);  
  
CREATE TABLE Salary_Audit (  
    audit_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_id INT,  
    old_salary INT,  
    new_salary INT,  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
INSERT INTO Employees (name, salary) VALUES  
('Pratham', 50000),  
('Shreya', 60000),  
('Vishwas', 55000);  
  
DELIMITER //  
  
CREATE TRIGGER After_Salary_Update  
AFTER UPDATE ON Employees  
FOR EACH ROW  
BEGIN  
    IF OLD.salary != NEW.salary THEN  
        INSERT INTO Salary_Audit (emp_id, old_salary, new_salary)  
        VALUES (OLD.id, OLD.salary, NEW.salary);  
    END IF;  
END //  
  
DELIMITER ;  
  
UPDATE Employees  
SET salary = 55000  
WHERE name = 'Pratham';  
  
UPDATE Employees  
SET salary = 65000
```

```
WHERE name = 'Shreya';
```

```
SELECT * FROM Salary_Audit;
```

Output :

Output:

audit_id	emp_id	old_salary	new_salary	changed_at
1	1	50000	55000	2026-09-29 06:26:48
2	2	60000	65000	2026-09-29 06:26:48

3.

```
CREATE TABLE Employees (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    dept VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Deleted_Employee_Log (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_id INT,  
    name VARCHAR(100),  
    dept VARCHAR(50),  
    deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
INSERT INTO Employees (name, dept) VALUES  
( 'Pratham', 'Management' ),  
( 'Shreya', 'Engineering' ),  
( 'Vishwas', 'Marketing' );
```

```
DELIMITER //
```

```
CREATE TRIGGER Before_Employee_Delete  
BEFORE DELETE ON Employees  
FOR EACH ROW  
BEGIN  
    IF OLD.dept = 'Management' THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Error: Employees in the Management department  
cannot be deleted!';  
    ELSE  
        INSERT INTO Deleted_Employee_Log (emp_id, name, dept)  
        VALUES (OLD.id, OLD.name, OLD.dept);  
    END IF;  
END //
```

```
DELIMITER ;
```

```
DELETE FROM Employees WHERE name = 'Shreya';
```

```
SELECT * FROM Deleted_Employee_Log;
```

```
SELECT * FROM Employees;
```

Output :

Output:

log_id	emp_id	name	dept	deleted_at
1	2	Shreya	Engineering	2026-09-29 06:28:29

id	name	dept
1	Pratham	Management
3	Vishwas	Marketing

procedures

1.

```
CREATE TABLE Students (  
    Student_ID INT PRIMARY KEY,  
    First_Name VARCHAR(50) NOT NULL,  
    Last_Name VARCHAR(50) NOT NULL,  
    Email VARCHAR(100),  
    Major VARCHAR(50),  
    Enrollment_Date DATE  
);  
  
INSERT INTO Students VALUES  
(101, 'pratham', 'bhat', 'pratham@example.com', 'Computer Science',  
'2024-08-15'),  
(102, 'shreya', 'N', 'shreya@example.com', 'Mathematics', '2024-08-16'),  
(103, 'haaha', 'hello', 'hah@example.com', 'Physics', '2024-08-18');  
  
DELIMITER //  
  
CREATE PROCEDURE GetStudentDetails(IN p_Student_ID INT)  
BEGIN  
    SELECT Student_ID, First_Name, Last_Name, Email, Major,  
    Enrollment_Date  
    FROM Students  
    WHERE Student_ID = p_Student_ID;  
END //  
  
DELIMITER ;  
  
CALL GetStudentDetails(101);
```

Output :

Output:

Student_ID	First_Name	Last_Name	Email	Major	Enrollment_Date
101	pratham	bhat	pratham@example.com	Computer Science	2024-08-15

